```
#include <iostream>
#include <string>
#include <map>
#include <vector>
#include <cstring>
#include <unistd.h>
#include <arpa/inet.h>

// Define the authentication states
enum AuthState {
    UNAUTHENTICATE,
    AUTHENTICATE,
    REGISTERED
};

// Define the function signature for
command handlers
typedef void (*CommandHandler)(int
client_fd, std::vector<std::string>& params);

// Define the Command structure with
```

```
function pointers
struct Command {
    std::string label;
    CommandHandler handler;  // Function
pointer for the handler
    AuthState requiredAuthState;
};

// Forward declarations of command
handlers
void handlePASS(int client_fd,
std::vector<std::string>& params);
void handleNICK(int client_fd,
std::vector<std::string>& params);
void handleUSER(int client_fd,
std::vector<std::string>& params);
void handleJOIN(int client_fd,
std::vector<std::string>& params);
void handlePART(int client_fd,
std::vector<std::string>& params);
void handleINVITE(int client_fd,
```

```
std::vector<std::string>& params);
void handleMODE(int client_fd,
std::vector<std::string>& params);
void handlePING(int client_fd,
std::vector<std::string>& params);
void handlePONG(int client_fd,
std::vector<std::string>& params);
void handlePRIVMSG(int client_fd,
std::vector<std::string>& params);
void handleKICK(int client_fd,
std::vector<std::string>& params);
void handleCAP(int client_fd,
std::vector<std::string>& params);
void handleQUIT(int client_fd,
std::vector<std::string>& params);
void handleWHO(int client_fd,
std::vector<std::string>& params);

// IRCServer class to handle the
connections and commands
class IRCServer {
```

```
private:
    int server_fd;
    std::map<int, AuthState>
clientAuthStates; // Maps client file
descriptor to authentication state
    std::map<std::string, Command>
commands; // Maps command label to
command structure

public:
    IRCServer(int port);
    ~IRCServer();
    void start();
    void handleClient(int client_fd);
    void sendToClient(int client_fd, const
std::string& message);
    void registerCommands();
    void parseCommand(int client_fd, const
std::string& line);
};
```

```
// Constructor to initialize the server
IRCServer::IRCServer(int port) {
    server_fd = socket(AF_INET,
SOCK_STREAM, 0);
    if (server_fd == -1) {
        perror("Socket creation failed");
        exit(EXIT_FAILURE);
    }

    struct sockaddr_in server_addr;
    memset(&server_addr, 0,
sizeof(server_addr));
    server_addr.sin_family = AF_INET;
    server_addr.sin_addr.s_addr =
INADDR_ANY;
    server_addr.sin_port = htons(port);

    if (bind(server_fd, (struct
sockaddr*)&server_addr,
sizeof(server_addr)) < 0) {
        perror("Bind failed");
```

```
        exit(EXIT_FAILURE);
    }

    if (listen(server_fd, 5) < 0) {
        perror("Listen failed");
        exit(EXIT_FAILURE);
    }

    registerCommands();
}

// Destructor to close the server socket
IRCServer::~IRCServer() {
    close(server_fd);
}

// Register all the available commands
with their handlers and required
authentication states
void IRCServer::registerCommands() {
    commands["PASS"] = {"PASS",
```

```
handlePASS, UNAUTHENTICATE};
    commands["NICK"] = {"NICK",
handleNICK, UNAUTHENTICATE};
    commands["USER"] = {"USER",
handleUSER, UNAUTHENTICATE};
    commands["JOIN"] = {"JOIN",
handleJOIN, REGISTERED};
    commands["PART"] = {"PART",
handlePART, REGISTERED};
    commands["INVITE"] = {"INVITE",
handleINVITE, REGISTERED};
    commands["MODE"] = {"MODE",
handleMODE, REGISTERED};
    commands["PING"] = {"PING",
handlePING, REGISTERED};
    commands["PONG"] = {"PONG",
handlePONG, REGISTERED};
    commands["PRIVMSG"] = {"PRIVMSG",
handlePRIVMSG, REGISTERED};
    commands["KICK"] = {"KICK",
handleKICK, REGISTERED};
```

```
    commands["CAP"] = {"CAP", handleCAP,
REGISTERED};
    commands["QUIT"] = {"QUIT",
handleQUIT, REGISTERED};
    commands["WHO"] = {"WHO",
handleWHO, REGISTERED};
}

// Send a message to a client
void IRCServer::sendToClient(int client_fd,
const std::string& message) {
    send(client_fd, message.c_str(),
message.length(), 0);
}

// Parse and dispatch the received
command
void IRCServer::parseCommand(int
client_fd, const std::string& line) {
    size_t space_pos = line.find(' ');
    std::string command = (space_pos ==
```

```
std::string::npos) ? line : line.substr(0,
space_pos);
    std::vector<std::string> params;

    // Parse the parameters
    while (space_pos != std::string::npos) {
        size_t next_space_pos = line.find(' ',
space_pos + 1);

params.push_back(line.substr(space_pos
+ 1, next_space_pos - space_pos - 1));
        space_pos = next_space_pos;
    }

    // Handle the command
    if (commands.find(command) !=
commands.end()) {
        Command cmd =
commands[command];

        if (clientAuthStates[client_fd] >=
```

```
cmd.requiredAuthState) {
            cmd.handler(client_fd, params);  //
Call the handler using the function pointer
        } else {
            sendToClient(client_fd, "ERROR:
Command requires authentication\n");
        }
    } else {
        sendToClient(client_fd, "ERROR:
Unknown command\n");
    }
}

// Handle a new client connection
void IRCServer::handleClient(int client_fd) {
    char buffer[1024];
    int read_size;

    while ((read_size = recv(client_fd, buffer,
sizeof(buffer) - 1, 0)) > 0) {
        buffer[read_size] = '\0'; // Null-
```

```
terminate the received data
        std::string line(buffer);
        parseCommand(client_fd, line);
    }

    if (read_size == 0) {
        close(client_fd);
    } else {
        perror("recv failed");
    }
}

// Start the server to accept and handle
client connections
void IRCServer::start() {
    std::cout << "Server started, waiting for
connections...\n";
    struct sockaddr_in client_addr;
    socklen_t client_addr_len =
sizeof(client_addr);
```

```
    while (true) {
        int client_fd = accept(server_fd, (struct
sockaddr*)&client_addr, &client_addr_len);
        if (client_fd < 0) {
            perror("Accept failed");
            continue;
        }
        std::cout << "New client connected\n";
        clientAuthStates[client_fd] =
UNAUTHENTICATE; // New client starts as
unauthenticated
        handleClient(client_fd);
    }
}

// Main function to start the IRC server
int main() {
    IRCServer server(6667); // Default IRC
port
    server.start();
    return 0;
```

```
}

// Command Handlers

void handlePASS(int client_fd,
std::vector<std::string>& params) {
    if (params.empty()) {
        sendToClient(client_fd, "ERROR: PASS
command requires a password\n");
        return;
    }
    std::string password = params[0]; //
Extract the password
    // Authentication logic here (for
example, check a stored password)
    sendToClient(client_fd, "PASS command
processed\n");
}

void handleNICK(int client_fd,
std::vector<std::string>& params) {
```

```
    if (params.empty()) {
        sendToClient(client_fd, "ERROR: NICK
command requires a nickname\n");
        return;
    }
    std::string nickname = params[0];
    sendToClient(client_fd, "NICK command
processed with nickname " + nickname +
"\n");
}

void handleUSER(int client_fd,
std::vector<std::string>& params) {
    if (params.size() < 4) {
        sendToClient(client_fd, "ERROR: USER
command requires 4 parameters\n");
        return;
    }
    std::string username = params[0];
    sendToClient(client_fd, "USER command
processed with username " + username +
```

```
"\n");
}

void handleJOIN(int client_fd,
std::vector<std::string>& params) {
    if (params.empty()) {
        sendToClient(client_fd, "ERROR: JOIN
command requires a channel name\n");
        return;
    }
    std::string channel = params[0];
    sendToClient(client_fd, "JOIN command
processed for channel " + channel + "\n");
}

void handlePART(int client_fd,
std::vector<std::string>& params) {
    if (params.empty()) {
        sendToClient(client_fd, "ERROR: PART
command requires a channel name\n");
        return;
```

```
    }
    std::string channel = params[0];
    sendToClient(client_fd, "PART command
processed for channel " + channel + "\n");
}

// Add other command handlers (INVITE,
MODE, PING, PONG, etc.) similarly
```